

mattpocock-skills

× Reversa

Relatório comparativo de dois sistemas de Agent Skills

Comparação entre o conjunto de skills de engenharia do Matt Pocock — objeto da engenharia reversa concluída em 30/07/2026 — e o framework Reversa, instalado no mesmo repositório. Os dois compartilham o mesmo substrato técnico e resolvem problemas opostos.

	mattpocock-skills	Reversa
O que é	Caixa de ferramentas de trabalho diário	Framework de engenharia reversa e SDD
Unidade	41 skills independentes	49 agentes sob um orquestrador
Sessão típica	Uma skill, uma tarefa, sem memória	Multi-sessão, com plano e retomada
Escrita	No repositório do usuário	Confinada a 4 pastas próprias
Idioma	Inglês	Português

Todos os números deste relatório foram medidos diretamente nos arquivos dos dois sistemas em 30/07/2026, não estimados. As contagens de skills, linhas, marcas de invocação e carga de contexto são reproduzíveis.

1. Resumo executivo

Os dois sistemas nascem da mesma constatação — que um agente de IA é estocástico e precisa de prosa normativa para se comportar de forma previsível — e divergem em tudo o que vem depois. A diferença de fundo não é de qualidade, é de formato do problema:

- O mattpocock-skills resolve problemas curtos e repetidos. Escrever um teste, diagnosticar um bug, quebrar uma spec em issues. Cada tarefa cabe numa sessão e termina nela.
- O Reversa resolve um problema longo e único. Ler 168 arquivos, cruzar 314 commits, produzir 97 documentos. Nenhuma sessão comporta isso.

Essa única diferença explica quase todas as outras. Um sistema que termina na sessão não precisa de estado, plano, checkpoint ou trilha de auditoria — e o mattpocock não tem nenhum dos quatro. Um sistema que atravessa sessões precisa dos quatro, e o Reversa tem todos.

O achado central deste relatório

O Reversa paga hoje ~3.677 tokens de carga permanente de contexto, contra ~890 do mattpocock-skills — 4,1× mais, com apenas 20% mais skills. A causa é única e tem correção mecânica: as 49 skills do Reversa são model-invoked, enquanto o mattpocock tornou 24 das suas 41 user-invoked de propósito. É a sugestão nº 1 da seção 7.

O que cada um faz melhor, em uma linha

Sistema	Vantagem decisiva	Por quê
Reversa	Trabalho que não cabe numa sessão	Estado persistente, plano editável, checkpoints e retomada — o mattpocock não tem equivalente estrutural
Reversa	Honestidade epistêmica	A escala CONFIRMADO / INFERIDO / LACUNA obriga cada afirmação a declarar sua origem
mattpocock	Economia de contexto	O eixo de invocação é tratado como decisão de engenharia, com custo medido e assumido
mattpocock	Portabilidade real entre harnesses	41 de 41 skills com marca dupla (Claude + Codex), 0 descasamentos

2. Os dois sistemas em números

Medição direta, 30/07/2026.

Métrica	mattpocock	Reversa	Leitura
Skills (SKILL.md)	41	49	Portes comparáveis
Arquivos totais	113	108	Praticamente idênticos
Linhas de conteúdo	4.560	10.802	Reversa 2,4× maior
Linhas só em SKILL.md	2.823	6.510	Reversa 2,3× maior
Média por SKILL.md	69 linhas	133 linhas	Reversa 1,9× mais longo
Menor / maior SKILL.md	7 / 140	65 / 328	mattpocock tem skills de 7 linhas; o Reversa não desce de 65

Métrica	mattpocock	Reversa	Leitura
Marca Codex (openai.yaml)	41 de 41	0 de 49	Reversa declara compatibilidade que não marca
User-invoked	24 de 41	0 de 49	O eixo de invocação não é usado no Reversa
Carga permanente de contexto	~890 tokens	~3.677 tokens	4,1x — consequência direta da linha acima
Estado persistente	nenhum	state.json + plan.md	Diferença estrutural
Escala de confiança	nenhuma	3 níveis	Só o Reversa qualifica afirmações
Pastas de referência	companheiros por skill	17 references/ + 7 templates/	Mesma ideia, nomes diferentes

Como a carga de contexto foi calculada

Somamos os caracteres do campo description de toda skill que NÃO tem disable-model-invocation (user-invoked não paga carga, por não ter description), dividido por 4 para estimar tokens. mattpocock: 17 descriptions, 3.562 caracteres. Reversa: 49 descriptions, 14.708 caracteres. É o custo pago em toda requisição, antes de qualquer trabalho começar.

3. Semelhanças

Os dois são o mesmo tipo de artefato, e isso é mais profundo do que a coincidência de formato.

1 · O mesmo substrato técnico

Ambos são Agent Skills: um diretório por skill, um SKILL.md com frontmatter YAML, corpo em markdown. Ambos rodam em Claude Code e declaram compatibilidade com outros harnesses. Nenhum dos dois tem código executável na trilha principal — nenhuma dependência de runtime, nenhum arquivo de teste.

2 · Lógica em prosa, executada por um agente estocástico

Esta é a semelhança de fundo. Nos dois sistemas, o que seria uma função é um parágrafo, e o que seria um if é uma frase condicional. Isso traz a mesma vantagem — a lógica é legível e editável por quem não programa — e o mesmo custo: nenhuma regra é garantida por mecanismo, todas dependem de o agente se autopolicar. A extração registrou cinco regras do mattpocock nessa condição, e o Reversa tem as suas (o portão bloqueante após o Scout, a regra de um agente por vez, a proibição de sobrescrever arquivos do legado).

3 · Divulgação progressiva por ponteiro

Os dois separam o que fica sempre carregado do que é buscado sob demanda, e alcançam o segundo por uma referência em prosa dentro do corpo. No mattpocock são os arquivos companheiros — 25 deles em 13 skills, do tipo MISSION-FORMAT.md. No Reversa são as 17 pastas references/ e 7 templates/. Mesmo mecanismo, mesma fragilidade: um ponteiro mal redigido, ou ausente, torna o alvo inalcançável. O mattpocock tem um caso documentado disso (GLOSSARY-FORMAT.md, órfão desde o commit de nascimento).

4 · Um ponto de entrada que mapeia o resto

O mattpocock tem o ask-matt, um router user-invoked cuja função é dizer qual skill usar. O Reversa tem o reversa-agents-help como catálogo e o próprio reversa como orquestrador. Os dois reconhecem que dezenas de skills soltas impõem ao humano o custo de saber quais existem.

5 · Instalação por um comando npx

npx skills add mattpocock/skills de um lado, npx reversa do outro. Os dois se distribuem como conteúdo copiado para a máquina do usuário, não como serviço.

4. Diferenças

Eixo	mattpocock-skills	Reversa
Propósito	Corrigir modos de falha do agente no trabalho diário	Extrair especificações de um sistema existente
Topologia	41 skills independentes, acopladas só por prosa	1 orquestrador + 48 agentes subordinados a um plano
Duração	Uma sessão, sem memória entre elas	Multi-sessão, com retomada explícita
Estado	Nenhum. Cada invocação começa do zero	state.json, plan.md, checkpoints por agente
Organização	6 buckets = ciclo de vida de promoção	49 agentes planos, agrupados por fase na prosa
Invocação	Eixo explícito: 24 user-invoked, 17 model-invoked	Todas model-invoked; o eixo não é usado
Portabilidade	Marca dupla obrigatória, mantida em lockstep	Marca única (frontmatter), apesar de declarar 4 harnesses
Saída	Efeito colateral no repositório do usuário	Artefatos em _reversa_sdd/, _reversa_docs/, _reversa_forward/
Rastreabilidade	Nenhuma. A skill afirma sem qualificar	Escala de confiança + matrizes + questions.md
Governança	CLAUDE.md com 12 invariantes declaradas	config.toml + state.json versionado
Configuração	Uma vez por repo, via setup-matt-pocock-skills	Por execução: doc_level, answer_mode, granularity
Segurança	Edita CLAUDE.md, cria docs/agents/, escreve no repo	Regra absoluta: nunca toca arquivo pré-existente

5. Pontos fortes de cada um

5.1 · mattpocock-skills

- Economia de contexto como decisão de engenharia. O eixo user-invoked / model-invoked não é detalhe de configuração: é o centro do desenho. 24 das 41 skills abrem mão de serem descobertas pelo agente para não custar nada quando não estão em uso. O glossário da casa nomeia os dois lados do trade-off — context load para a máquina, cognitive load para o humano — e trata o segundo como preço legítimo da agência humana, não como defeito.

- Poda. Média de 69 linhas por SKILL.md, com skills de 7 linhas que só apontam para outra. Existe uma skill inteira (writing-great-skills) dedicada a catalogar os modos de falha de escrever skills, com glossário formal e a convenção `_Avoid_` para termos rejeitados.
- Ciclo de promoção. Os buckets não são pastas de organização: são estados de maturidade. Uma skill vive em `in-progress/` enquanto é rascunho, sobe para `engineering/` quando amadurece, e cai em `deprecated/` quando sai de uso — e o histórico mostra o ciclo completo funcionando, inclusive a saída definitiva do repositório.
- Portabilidade verificada. 41 de 41 skills carregam a marca de invocação nos dois formatos (frontmatter do Claude e `agents/openai.yaml` do Codex), com zero descasamentos medidos.
- Base de conhecimento de recusas. A pasta `.out-of-scope/` registra o que o projeto decidiu NÃO fazer, com o argumento e o número do pedido original. É o inverso do backlog: um registro de fronteiras que evita rediscutir a mesma proposta.
- Vocabulário formalizado. Um glossário opinativo define os termos do domínio com o que evitar em cada caso, e é carregado só quando o assunto exige.

5.2 · Reversa

- Estado persistente e retomada. `state.json` guarda fase, checkpoints por agente e o que falta; `plan.md` é uma lista de tarefas marcável. Um trabalho de dezenas de horas sobrevive ao fim da janela de contexto — e sobreviveu: esta comparação existe porque a extração foi retomada dias depois exatamente de onde parou.
- Escala de confiança. Toda afirmação declara se foi extraída do código, inferida de padrões ou é lacuna. É a diferença entre um documento que afirma e um documento auditável — e permitiu que a própria extração encontrasse quatro erros seus.
- Plano como contrato. O `plan.md` é gerado antes da execução e pode ser editado pelo usuário: adicionar, remover, reordenar tarefas. O que vai rodar é negociado antes, não descoberto depois.
- Profundidade configurável. `doc_level` (essencial / completo / detalhado) muda o comportamento de vários agentes de uma vez — quais artefatos gerar, se a revisão cruzada é opcional ou obrigatória, se as lacunas são categorizadas por severidade. Uma decisão do usuário, propagada.
- Regra absoluta de não-destruição. O Reversa escreve apenas em `.reversa/`, `_reversa_sdd/`, `_reversa_docs/` e `_reversa_forward/`. Nunca apaga nem sobrescreve arquivo do projeto analisado. É uma garantia enunciada como inegociável e repetida em cada skill.
- Cobertura de ciclo completo. Seis fluxos que se conectam — descobrir, criar do zero, implementar, migrar, documentar, consultar o catálogo — contra um conjunto de ferramentas que o usuário encadeia manualmente.
- Verificação de regressão semântica. Em re-extrações, compara o que foi codado no ciclo forward contra os artefatos recém-gerados e emite veredito por item. Não há nada análogo do outro lado.

6. Onde o Reversa é melhor, e por quê

Esta seção isola as vantagens em que o Reversa não é apenas diferente, mas superior — e explica a causa de cada uma. O padrão que emerge é consistente: o Reversa é melhor em tudo que decorre de aceitar que o trabalho é longo, incerto e precisa ser auditável.

6.1 · Trabalho que não cabe numa sessão

O mattpocock tem uma skill chamada handoff, que comprime a conversa num arquivo markdown para você abrir outra sessão e continuar. É uma boa ferramenta e é tudo o que existe: a continuidade depende de o humano lembrar de invocá-la antes de a janela estourar, e o que sobrevive é um resumo em prosa.

O Reversa trata continuidade como propriedade do sistema. O estado é salvo em marcos definidos, os checkpoints registram o que cada agente produziu, e há um passo de retomada que lê o estado e continua. Há inclusive um checkpoint preventivo: o orquestrador oferece a pausa antes de um agente pesado, em vez de esperar o contexto estourar.

Por que é melhor: porque não depende de disciplina humana no momento de maior pressão. A diferença não é de recurso, é de quem carrega a responsabilidade — o humano ou o sistema.

6.2 · Honestidade epistêmica embutida

Uma skill do mattpocock afirma sem qualificar. Quando o triage classifica uma issue ou o domain-modeling propõe um termo, o resultado não vem marcado com o grau de certeza — o usuário avalia na hora, o que funciona porque ele está lendo o resultado imediatamente.

O Reversa força cada afirmação a declarar sua origem: extraída do código, inferida de padrões, ou lacuna que exige validação humana. Isso muda a natureza do artefato produzido. Um documento em que 65% das afirmações são confirmadas e 16% são lacunas declaradas é utilizável por alguém que não acompanhou a produção; um documento que afirma tudo com a mesma voz não é.

Por que é melhor: porque o produto do Reversa é consumido depois, por outra pessoa ou por outro agente. Sem a escala, o consumidor não tem como separar o que foi medido do que foi adivinhado. A prova está no próprio relatório de confiança: a escala permitiu localizar quatro afirmações erradas da extração — algo impossível num documento sem gradação.

6.3 · Orquestração determinística contra encadeamento por memória

No mattpocock, a sequência entre skills é prosa: o ask-matt sugere que depois do to-spec você rode o to-tickets, e a página de docs de uma skill menciona a próxima. O encadeamento existe na cabeça do usuário e no texto, não no sistema. Se ele esquecer um passo, nada avisa.

No Reversa, a sequência é um arquivo. O plano é gerado, apresentado, editável, e a execução marca cada item concluído. O orquestrador sabe qual é a próxima tarefa porque ela está escrita.

Por que é melhor: um pipeline de seis fases com dependências entre elas não pode depender de memória. E há um ganho que o mattpocock não tem como oferecer — o usuário revisa e altera o plano antes de qualquer trabalho começar.

6.4 · Auditabilidade do próprio processo

O Reversa produz a trilha do que fez: quais arquivos cada agente leu, quais achados registrou, quais perguntas gerou, que avisos acumulou ao rodar em modo autônomo. É possível reconstruir por que uma spec diz o que diz.

As skills do mattpocock não deixam rastro. Depois que o to-tickets criou as issues, não há registro do raciocínio que levou àquele recorte.

Por que é melhor: engenharia reversa é uma cadeia de inferências sobre um sistema que ninguém explicou. Sem trilha, o resultado é inaudível — e um documento de especificação inaudível é um documento que ninguém pode contestar, o que é pior do que um errado.

6.5 · Segurança por desenho

A regra do Reversa é enunciada como inegociável e repetida em cada skill: nunca apagar, modificar ou sobrescrever arquivo pré-existente; a escrita fica confinada a quatro pastas próprias. Um usuário pode rodar o Reversa num repositório que não conhece sem risco de dano.

O mattpocock, por natureza, edita o repositório do usuário — CLAUDE.md, docs/agents/, issues. É adequado ao propósito, mas é uma superfície de escrita muito maior e sem fronteira declarada.

Por que é melhor: no contexto de analisar sistema alheio, uma fronteira de escrita explícita e estreita é uma garantia que o usuário pode verificar sem ler todo o código.

6.6 · Adaptação de profundidade ao projeto

O doc_level é escolhido uma vez e propaga por toda a execução, alterando o comportamento de vários agentes. O mattpocock não tem eixo equivalente: cada skill tem a profundidade que tem.

Por que é melhor: o mesmo pipeline serve um script de 200 linhas e um sistema legado de milhares de arquivos, sem forçar o usuário a escolher entre superficial demais e caro demais.

7. Sugestões do mattpocock para o Reversa

Em ordem de impacto. As duas primeiras têm ganho mensurável e custo mecânico.

Prioridade CRÍTICA · 1 · Usar o eixo de invocação

Hoje as 49 skills do Reversa são model-invoked. Cada uma mantém a description permanentemente no contexto do agente: 14.708 caracteres, ~3.677 tokens, pagos em toda requisição antes de qualquer trabalho. O mattpocock, com 41 skills, paga ~890 — porque tornou 24 delas user-invoked.

A observação decisiva é que a maioria dos agentes reversa-* nunca precisa ser descoberta por ninguém. Eles não são invocados pelo humano nem escolhidos pelo modelo: são chamados pelo orquestrador, por nome, a partir do plano. Uma description existe para tornar a skill auto-descobrável — e essas não precisam ser descobertas.

Proposta: manter model-invoked apenas os pontos de entrada que o usuário digita — reversa, reversa-new, reversa-forward, reversa-migrate, reversa-docs, reversa-agents-help — e marcar os outros ~43 com disable-model-invocation: true.

	Hoje	Depois	Ganho
Skills com description carregada	49	~6	–88%
Caracteres em contexto	14.708	~1.900	–87%
Tokens permanentes	~3.677	~475	–3.200 tokens

Ressalva honesta sobre esta sugestão

Uma skill user-invoked não pode ser invocada por outra skill — sem description, nada além do humano a alcança. Isso quebraria a instrução "Ative o skill reversa-[agente]" do orquestrador. Mas o próprio SKILL.md do reversa já traz o caminho alternativo: "Se a engine não suportar ativação direta de skills por nome, leia .agents/skills/reversa-[agente]/SKILL.md na íntegra e execute no contexto atual." Ler o arquivo funciona independentemente da marca. A migração é viável, mas exige tornar a leitura direta o caminho primário, não o fallback.

Prioridade ALTA · 2 · Adicionar a segunda marca de invocação

O frontmatter de cada skill do Reversa declara: compatibilidade com Claude Code, Codex, Cursor e Gemini CLI. Mas há 0 arquivos agents/openai.yaml em 49 skills. A política de invocação não atravessa para o Codex — a compatibilidade é declarada e não é marcada.

O mattpocock resolveu isso com uma regra que a extração registrou como ADR: um eixo conceitual, duas marcas físicas, mantidas em lockstep. 41 de 41 skills marcadas nos dois formatos, zero descasamentos. E o custo foi baixo — as 41 marcas foram adicionadas num único commit em lote.

Ganha relevância se a sugestão 1 for adotada: sem o openai.yaml, a economia de contexto valeria só no Claude Code.

Prioridade MÉDIA · 3 · Buckets como ciclo de promoção

O Reversa é plano: 49 agentes no mesmo nível, todos implicitamente prontos para produção. O mattpocock separa por maturidade — rascunhos em in-progress/, produção em engineering/ e productivity/, aposentados em deprecated/ — e o que não está promovido não chega ao usuário.

Para o Reversa, isso permitiria desenvolver um agente novo dentro do repositório sem que ele apareça no catálogo nem seja instalado, e aposentar um agente preservando-o como registro. Com 49 agentes e uma lista de instalados no config.toml, a separação por maturidade já é uma necessidade latente.

Prioridade MÉDIA · 4 · Poda

O SKILL.md médio do Reversa tem 133 linhas contra 69 do mattpocock, e o maior chega a 328 — quase 2,5× o maior do outro lado. Skills longas custam atenção do agente, não só tokens, e a parte do meio de um documento longo é a que o modelo mais ignora.

O mattpocock tem uma skill inteira sobre isso, com os modos de falha catalogados e nomeados — duplicação, sedimento, ponteiro fraco. Vale rodá-la sobre os cinco maiores SKILL.md do Reversa: boa parte do conteúdo longo é candidato natural a virar arquivo em references/, alcançado por ponteiro, em vez de ficar sempre carregado.

Prioridade MÉDIA · 5 · Registrar as recusas

A pasta `.out-of-scope/` do `mattpocock` guarda o que o projeto decidiu não fazer, com o argumento completo e o número do pedido que originou a discussão. Três documentos hoje.

Para um framework com usuários pedindo recursos, é o mecanismo mais barato de evitar rediscutir a mesma proposta a cada trimestre — e de responder um pedido com uma página em vez de uma conversa. O valor não está na recusa, está no argumento preservado.

Prioridade BAIXA · 6 · Glossário do próprio vocabulário

O `Reversa` tem vocabulário próprio e específico — `unit`, `spec`, `lacuna`, `fase`, `checkpoint`, `doc_level`, `granularity`, `agente independente` — que hoje vive espalhado entre o `SKILL.md` do orquestrador, os `references/` e o `config.toml`. O `mattpocock` formaliza o seu num glossário com definição opinativa e lista do que evitar em cada termo.

Com 49 agentes escritos ao longo do tempo, é o mecanismo que impede que dois deles usem palavras diferentes para a mesma coisa.

Prioridade BAIXA · 7 · Declarar as invariantes — mas não copiar o erro

O `mattpocock` declara 12 invariantes estruturais no `CLAUDE.md`, ligando `skill`, `README`, `manifesto`, `docs` e `versão`. É um bom padrão: torna explícito o que precisa mudar junto.

Copie a declaração, não a ausência de executor

Nenhuma das 12 invariantes do `mattpocock` é verificada por processo — o único CI é o de `release`, que não valida estrutura. Duas estão quebradas neste momento, e ambas são exatamente do tipo que uma verificação de trinta linhas pegaria. A extração concluiu que a ausência é decisão deliberada, mas registrou que o custo já foi pago duas vezes. Se o `Reversa` adotar invariantes declaradas, vale adotá-las com um verificador — declarar sem verificar produz documentação que envelhece em silêncio.

8. Conclusão

Os dois sistemas estão bem desenhados para problemas diferentes, e a comparação direta de qualidade não é útil. O que é útil é notar que cada um resolveu bem exatamente aquilo que o outro não precisou resolver.

O `mattpocock-skills` otimizou para o que se usa muitas vezes: custo de contexto, concisão, portabilidade, um ciclo de vida que permite experimentar sem poluir o produto. São as virtudes de quem paga o preço todo dia.

O `Reversa` otimizou para o que se usa poucas vezes e não pode falhar: estado, plano, auditabilidade, gradação de confiança, fronteira de escrita. São as virtudes de quem precisa entregar um artefato que outra pessoa vai consumir sem ter acompanhado a produção.

A recomendação prática é estreita e concreta. As duas primeiras sugestões da seção 7 — usar o eixo de invocação e adicionar a marca do `Codex` — são mecânicas, têm ganho medido e não alteram nada do que o `Reversa` faz de melhor. Juntas, reduziriam a carga permanente de contexto em cerca de 87% e tornariam real uma compatibilidade que hoje está apenas declarada.